

Ecole Marocaine des Sciences de l'Ingénieur
EMSI

RAPPORT DE MINI PROJET

THÈME

**Système de Gestion des Votes
sur Blockchain**

FILIÈRE

Blockchain

RÉALISÉ PAR

**Farhi Naima
Anejjar Rachida
Redouani Mohamed Reda**

ENCADRANT ACADÉMIQUE

M. Khalid EL KHADIRI

Année Universitaire 2025 - 2026

Dédicaces

Nous dédions ce travail à nos familles, pour leur soutien indéfectible, leur patience et leurs encouragements tout au long de notre parcours universitaire.

À nos professeurs de l'EMSI, qui ont su nous transmettre les connaissances et la rigueur nécessaires pour mener à bien ce projet.

À tous ceux qui, de près ou de loin, ont contribué à notre formation et à l'aboutissement de ce mini projet.

Farhi Naima, Anejjar Rachida, Redouani Mohamed Reda

Résumé

Ce rapport présente la conception et le développement d'un système de gestion des votes basé sur la technologie Blockchain, réalisé dans le cadre du cours Blockchain à l'Ecole Marocaine des Sciences de l'Ingénieur (EMSI).

L'objectif principal du projet est de démontrer comment la Blockchain peut garantir la transparence, l'immutabilité et la sécurité d'un processus de vote électronique, sans recourir à une autorité centrale de confiance. La logique métier est entièrement encapsulée dans un smart contract écrit en Solidity et déployé sur un réseau Ethereum local simulé par Ganache.

L'application comprend une interface web permettant à un administrateur de gérer les candidats et les sessions de vote, et à des électeurs de voter de manière sécurisée via leur adresse Ethereum. Le projet a été développé avec Truffle, Ganache, Web3.js et Node.js, dans un environnement de développement conteneurisé via Docker et VS Code Dev Containers.

Mots-clés : Blockchain, Smart Contract, Solidity, Ethereum, Ganache, Truffle, Web3.js, Vote Électronique, DApp, Docker.

Abstract

This report presents the design and development of a blockchain-based voting management system, carried out as part of the Blockchain course at the Ecole Marocaine des Sciences de l'Ingénieur (EMSI).

The main objective is to demonstrate how Blockchain technology can ensure transparency, immutability, and security in an electronic voting process, without relying on a central trusted authority. The business logic is entirely encapsulated in a Solidity smart contract deployed on a local Ethereum network simulated by Ganache.

The application features a web interface allowing an administrator to manage candidates and voting sessions, and voters to cast their votes securely using their Ethereum address. The project was built with Truffle, Ganache, Web3.js and Node.js, within a containerized development environment using Docker and VS Code Dev Containers.

Keywords: Blockchain, Smart Contract, Solidity, Ethereum, Ganache, Truffle, Web3.js, Electronic Voting, DApp, Docker.

Liste des Abréviations

Abréviation	Description
ABI	Application Binary Interface — interface de description des fonctions d'un smart contract
DApp	Decentralized Application — application décentralisée fonctionnant sur une blockchain
EVM	Ethereum Virtual Machine — machine virtuelle exécutant les smart contracts Ethereum
ETH	Ether — cryptomonnaie native du réseau Ethereum
IDE	Integrated Development Environment — environnement de développement intégré
JSON	JavaScript Object Notation — format d'échange de données léger
RPC	Remote Procedure Call — appel de procédure à distance
VS Code	Visual Studio Code — éditeur de code source développé par Microsoft
Web3	Ensemble de technologies permettant d'interagir avec des blockchains Ethereum

Table des Matières

Introduction Générale

CHAPITRE 1 : Contexte et Objectifs

1. Contexte du projet
2. Problématique
3. Objectifs
4. Technologies utilisées

CHAPITRE 2 : Analyse et Conception

1. Spécifications fonctionnelles
2. Architecture du système
3. Diagramme de cas d'utilisation
4. Diagramme de séquence
5. Smart Contract : structure et logique

CHAPITRE 3 : Réalisation

1. Environnement de développement
2. Développement du Smart Contract
3. Déploiement avec Truffle et Ganache
4. Interface utilisateur
5. Tests automatisés

CHAPITRE 4 : Conclusion et Perspectives

1. Bilan du projet
2. Difficultés rencontrées
3. Perspectives

Introduction Générale

Dans un monde de plus en plus numérisé, les systèmes de vote traditionnels font face à de nombreux défis : manque de transparence, risque de fraude, dépendance à des tiers de confiance et difficulté de vérification des résultats. Ces problèmes soulèvent des questions fondamentales sur l'intégrité des processus démocratiques.

La technologie Blockchain apporte une réponse naturelle et innovante à ces enjeux. Grâce à ses propriétés fondamentales — décentralisation, immuabilité, transparence et exécution automatique des règles via des smart contracts — elle permet de concevoir un système de vote infalsifiable, vérifiable par tous et ne nécessitant aucune autorité centrale.

C'est dans ce contexte que s'inscrit notre mini projet, réalisé dans le cadre du cours Blockchain à l'EMSI. Notre équipe a conçu et développé un système complet de gestion des votes sur Blockchain, combinant un smart contract Solidity déployé sur un réseau Ethereum local, une interface web multi-pages et une suite de tests automatisés.

Le présent rapport est organisé en quatre chapitres. Le premier chapitre présente le contexte, la problématique et les objectifs du projet ainsi que les technologies utilisées. Le deuxième chapitre est consacré à l'analyse et la conception du système. Le troisième chapitre détaille la réalisation technique et l'interface utilisateur. Enfin, le quatrième chapitre dresse le bilan du projet et trace les perspectives d'évolution.

CHAPITRE 1 :

CONTEXTE ET OBJECTIFS DU PROJET

« Ce chapitre introduit le cadre général du projet en présentant la problématique à laquelle nous avons souhaité répondre, les objectifs fixés, ainsi que les technologies choisies pour y parvenir. »

1. Contexte du projet

Ce mini projet a été réalisé dans le cadre du cours Blockchain dispensé à l'Ecole Marocaine des Sciences de l'Ingénieur (EMSI), durant l'année universitaire 2025-2026, au mois de mai 2026. Il a été développé par une équipe de trois étudiants sous la supervision de Monsieur Khalid EL KHADIRI.

L'objectif pédagogique est de mettre en pratique les concepts fondamentaux de la Blockchain — smart contracts, transactions, décentralisation — à travers un cas d'usage concret : la gestion d'un système de vote électronique.

Projet	Système de Gestion des Votes sur Blockchain
Équipe	Farhi Naima, Anejjar Rachida, Redouani Mohamed Reda
Encadrant	M. Khalid EL KHADIRI
École	EMSI — Ecole Marocaine des Sciences de l'Ingénieur
Cours	Blockchain
Période	Mai 2026
Dépôt GitHub	https://github.com/NaimaFarhi/Voting-System

2. Problématique

Les systèmes de vote classiques souffrent de plusieurs limites majeures :

- Manque de transparence : les résultats peuvent être manipulés sans que les électeurs ne puissent les vérifier.
- Dépendance à une autorité centrale : l'intégrité du vote repose entièrement sur la confiance accordée à un organisme tiers.
- Risque de double vote : sans mécanisme robuste, un électeur peut voter plusieurs fois.
- Absence de traçabilité immuable : une fois le vote terminé, il est difficile d'auditer l'historique des opérations.

Comment concevoir un système de vote transparent, infalsifiable et sans autorité centrale, tout en garantissant qu'un électeur ne puisse voter qu'une seule fois ?

3. Objectifs

- Développer un smart contract Solidity gérant la logique complète du vote on-chain.
- Déployer le contrat sur un réseau Ethereum local simulé par Ganache.
- Créer une interface web multi-pages (page d'accueil, panel admin, page électeur).
- Garantir qu'un même compte Ethereum ne peut voter qu'une seule fois.
- Séparer les rôles : l'administrateur gère les candidats et les sessions, les électeurs votent.
- Automatiser le déploiement et les tests avec Truffle.
- Assurer la portabilité de l'environnement via Docker et VS Code Dev Containers.

4. Technologies utilisées

Technologie	Rôle	Version
Solidity	Langage de programmation des smart contracts	^0.8.0
Truffle	Framework de compilation, déploiement et tests	v5.11.5
Ganache	Simulateur de blockchain Ethereum locale	v7.9.2
Web3.js	Librairie JavaScript d'interaction avec Ethereum	v1.10.0
Node.js	Environnement d'exécution JavaScript côté serveur	v18.20.8
Docker	Conteneurisation de l'environnement de développement	—
VS Code Dev Container	Environnement de développement portable	—
Git / GitHub	Versionnage et collaboration	—

CHAPITRE 2 :

ANALYSE ET CONCEPTION

« Ce chapitre présente l'analyse fonctionnelle du système, son architecture générale, ainsi que les diagrammes UML modélisant les interactions et les flux du processus de vote. »

1. Spécifications fonctionnelles

Le système doit satisfaire les besoins fonctionnels suivants :

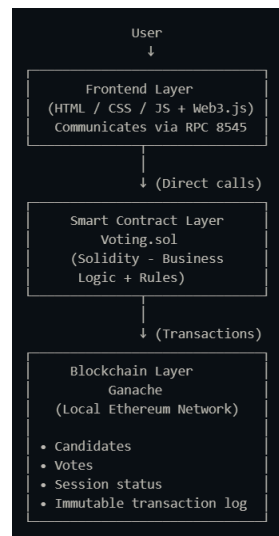
Fonctionnalité	Description	Acteur
Sélection de compte	L'utilisateur sélectionne son compte Ethereum parmi les comptes Ganache disponibles. Le système le redirige vers la page appropriée (admin ou électeur).	Tout utilisateur
Ajout de candidat	L'administrateur saisit le nom d'un candidat et l'ajoute à la liste enregistrée on-chain. Les noms vides sont rejetés.	Administrateur
Ouverture du vote	L'administrateur ouvre la session de vote. Impossible si le vote est déjà ouvert.	Administrateur
Fermeture du vote	L'administrateur clôture la session de vote. Impossible si le vote est déjà fermé.	Administrateur
Consultation des résultats	L'administrateur consulte en temps réel le nombre de votes par candidat avec des barres de progression.	Administrateur
Vote	L'électeur sélectionne un candidat et soumet son vote. Le vote est rejeté si la session est fermée ou si l'électeur a déjà voté.	Électeur
Confirmation de vote	Après avoir voté, l'électeur voit un message de confirmation. Il ne peut plus accéder au formulaire de vote.	Électeur
Déconnexion	Tout utilisateur peut se déconnecter, ce qui supprime la session et revient à la page d'accueil.	Tout utilisateur

2. Architecture du système

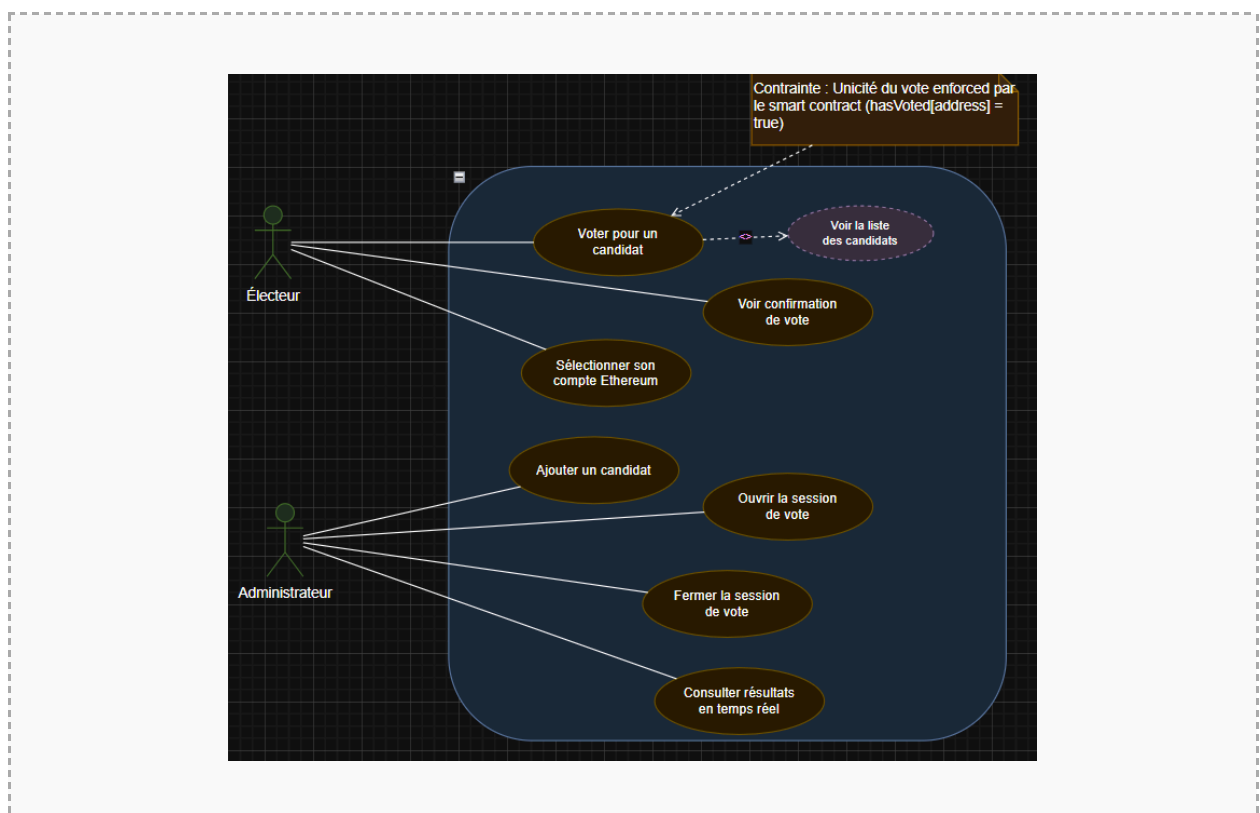
L'application suit une architecture 3 couches :

- Couche Blockchain (Ganache) : stocke l'état du contrat — candidats, votes enregistrés, statut de la session. Toute modification d'état est une transaction enregistrée dans un bloc immuable.
- Couche Smart Contract (Voting.sol) : contient toute la logique métier en Solidity. C'est la source de vérité absolue — aucun composant ne peut contourner ses règles.

- Couche Frontend (HTML / JS / Web3.js) : interface utilisateur qui communique directement avec le contrat via Web3.js sur le port RPC de Ganache (8545). Aucun backend serveur traditionnel n'est nécessaire.



3. Diagramme de cas d'utilisation

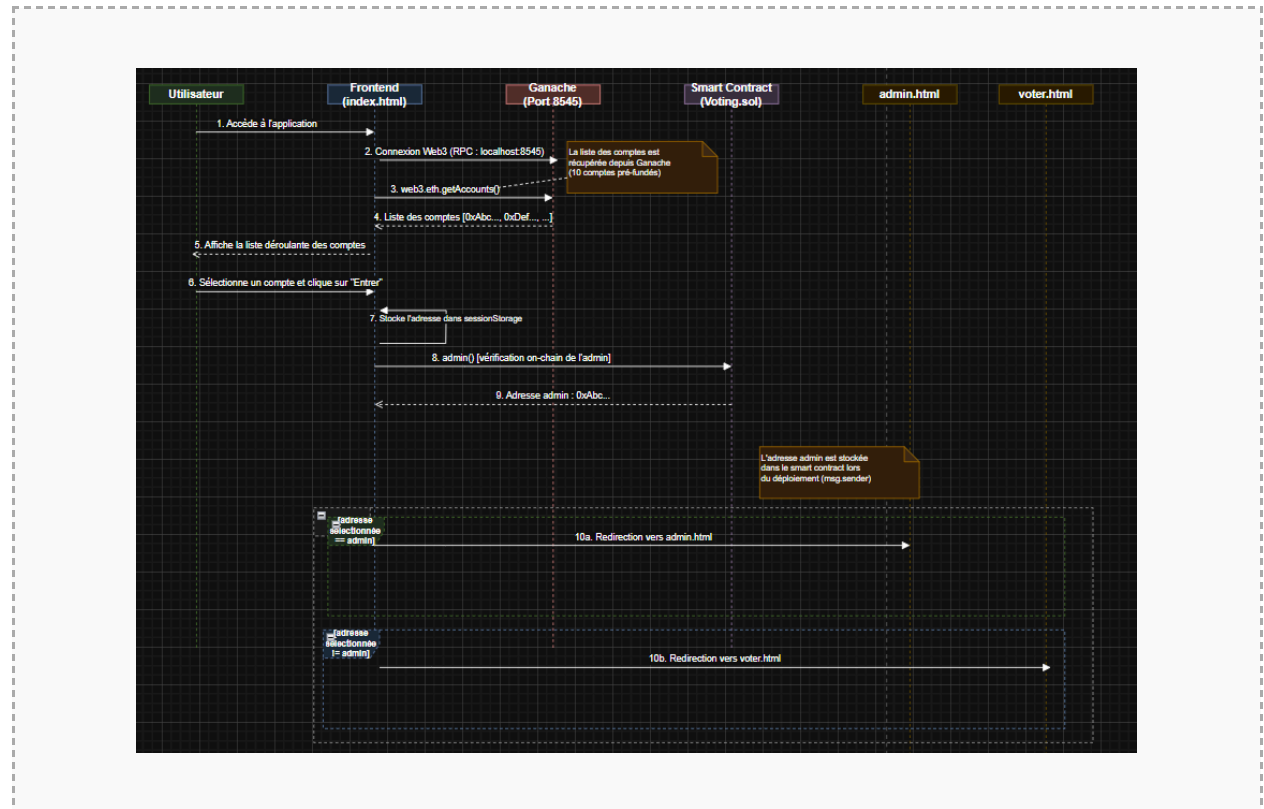


Le diagramme ci-dessus illustre les interactions entre les deux acteurs principaux du système — l'Administrateur et l'Électeur — et les fonctionnalités offertes par le système. L'administrateur dispose d'un accès exclusif aux fonctions de gestion (ajout de candidats,

contrôle de la session), tandis que les électeurs peuvent uniquement consulter les candidats et voter. La contrainte d'unicité du vote est enforced au niveau du smart contract.

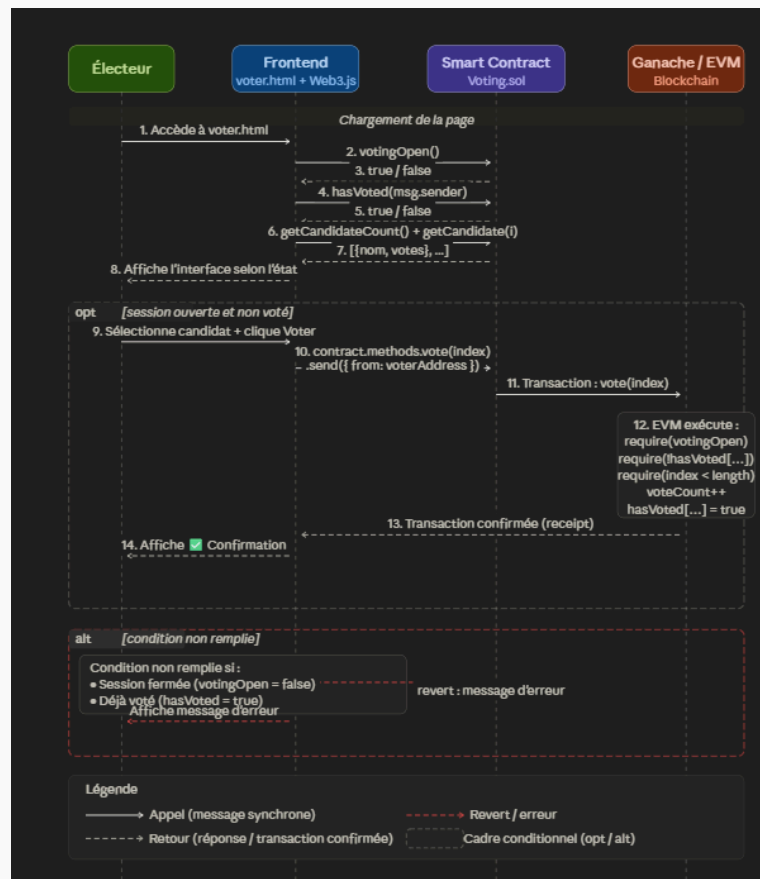
4. Diagramme de séquence

4.1 Séquence de connexion et redirection



Lors de l'accès à l'application, le frontend se connecte à Ganache sur le port 8545 et récupère la liste des comptes disponibles. L'utilisateur sélectionne son compte. Le système compare l'adresse sélectionnée avec l'adresse de l'administrateur (account[0] du déploiement). Si les adresses correspondent, il est redirigé vers le panel admin ; sinon, vers la page électeur.

4.2 Séquence de vote



Lorsqu'un électeur soumet son vote, le frontend appelle la fonction `vote(index)` du smart contract via `Web3.js`. Le contrat vérifie trois conditions : (1) la session de vote est ouverte, (2) l'adresse n'a pas encore voté, (3) l'index du candidat est valide. Si toutes les conditions sont remplies, le vote est enregistré on-chain et le mapping `hasVoted[adresse]` est mis à `true`. En cas d'échec d'une condition, la transaction est rejetée (`revert`) et le frontend affiche le message d'erreur correspondant.

5. Smart Contract : structure et logique

Le smart contract `Voting.sol` est le cœur du système. Il expose les structures de données et fonctions suivantes :

Élément	Type	Description
Candidate	struct	Stocke le nom (string) et le compteur de votes (uint) d'un candidat
candidates	Candidate[]	Tableau dynamique de tous les candidats enregistrés
admin	address	Adresse du compte déployeur, seul autorisé à administrer

votingOpen	bool	Indique si la session de vote est actuellement ouverte
hasVoted	mapping(address=>bool)	Enregistre les adresses ayant déjà exercé leur vote
addCandidate(name)	function	Ajoute un candidat. Réservé à l'admin. Rejette les noms vides.
openVoting()	function	Ouvre la session de vote. Rejette si déjà ouverte.
closeVoting()	function	Ferme la session de vote. Rejette si déjà fermée.
vote(index)	function	Enregistre le vote. Vérifie : session ouverte, pas encore voté, index valide.
getCandidate(index)	view	Retourne le nom et le nombre de votes d'un candidat.
getCandidateCount()	view	Retourne le nombre total de candidats.
onlyAdmin	modifier	Restreint l'accès à l'administrateur uniquement.
whenOpen	modifier	Restreint l'accès aux périodes où le vote est ouvert.

CHAPITRE 3 :

RÉALISATION

« Ce chapitre présente la mise en œuvre concrète du projet : l'environnement de développement, le code du smart contract, le déploiement sur Ganache, l'interface utilisateur et les tests automatisés. »

1. Environnement de développement

L'un des enjeux pratiques du projet était d'assurer un environnement identique sur les différentes machines des membres de l'équipe. Pour cela, nous avons opté pour un Dev Container VS Code basé sur Docker.

1.1 Dev Container

Le fichier `.devcontainer/devcontainer.json` définit l'image Docker utilisée (Node.js 18), installe automatiquement Truffle et Ganache au démarrage du conteneur, configure le forwarding des ports nécessaires (8545 pour Ganache, 3000 pour le serveur frontend) et installe l'extension VS Code Solidity.

Ainsi, pour rejoindre le projet, un membre de l'équipe n'a besoin que de Docker Desktop et VS Code. Il clone le dépôt GitHub, ouvre le dossier dans VS Code et clique sur "Reopen in Container". Tout l'environnement est automatiquement configuré.

1.2 Structure du projet

Fichier / Dossier	Rôle
<code>.devcontainer/devcontainer.json</code>	Configuration du Dev Container Docker
<code>contracts/Voting.sol</code>	Smart contract principal (Solidity)
<code>migrations/2_deploy_voting.js</code>	Script de déploiement Truffle
<code>test/voting.test.js</code>	Suite de tests automatisés (19 tests)
<code>scripts/interact.js</code>	Script d'interaction CLI avec le contrat
<code>client/index.html</code>	Page d'accueil / sélection de compte
<code>client/admin.html</code>	Interface administrateur
<code>client/voter.html</code>	Interface électeur
<code>client/contracts/Voting.json</code>	ABI + adresse du contrat (généré par Truffle)
<code>truffle-config.js</code>	Configuration Truffle (réseau, compilateur)
<code>package.json</code>	Dépendances Node.js + scripts npm
<code>.gitignore</code>	Exclusion de <code>node_modules</code> , <code>build</code> , <code>.env</code>

2. Développement du Smart Contract

Le smart contract Voting.sol a été développé en Solidity ^0.8.0. Toute la logique métier est encapsulée dans ce fichier unique. Le contrat suit les bonnes pratiques Solidity : utilisation de modifiers pour les gardes d'accès, vérification systématique des préconditions avec require(), commentaires NatSpec (///) pour la documentation, et séparation claire entre les fonctions d'écriture (transactions) et de lecture (view).

Les règles métier les plus critiques sont enforcées au niveau du contrat lui-même, et non au niveau du frontend. Cela garantit qu'elles ne peuvent pas être contournées, quelle que soit l'interface utilisée pour interagir avec le contrat :

- Unicité du vote : le mapping hasVoted[msg.sender] empêche tout double vote par la même adresse.
- Contrôle d'accès admin : le modifier onlyAdmin vérifie que msg.sender == admin avant toute fonction administrative.
- Intégrité de la session : le modifier whenOpen empêche de voter en dehors d'une session ouverte.
- Validation des données : les noms de candidats vides sont rejetés, les index hors limites sont vérifiés.

3. Déploiement avec Truffle et Ganache

Ganache simule un réseau Ethereum complet en local, avec 10 comptes pré-financés en ETH fictif. Pour garantir des adresses stables entre les redémarrages, Ganache est lancé avec un mnémonique fixe. Le compte accounts[0] est toujours l'administrateur, car c'est lui qui déploie le contrat (msg.sender dans le constructeur).

Truffle compile le contrat Solidity, génère l'ABI et le bytecode, puis déploie le contrat sur le réseau Ganache via le script de migration. L'adresse du contrat déployé est automatiquement injectée dans le fichier client/contracts/Voting.json via la commande npm run copy-abi, évitant tout hardcoding d'adresse dans les fichiers HTML.

Résultat d'un déploiement type :


```

node →/workspaces/voting-system (main) $ truffle migrate --network development --reset

Compiling your contracts...
=====
> Everything is up to date, there is nothing to compile.

Starting migrations...
=====
> Network name:    'development'
> Network id:     1778354087524
> Block gas limit: 30000000 (0xc9c380)

2 deploy_voting.js
=====

Deploying 'Voting'
-----
> transaction hash: 0x28d96e79db140fe6f3acfd473ea7ce9cb123642a8de1637028db9ad3954f667
> Blocks: 0
> contract address: 0x6b83f0e57898A64C67cBdBF79F4Ee905770efd72
> block number: 1
> block timestamp: 1778354121
> account: 0x91ef41A588F24AF8FD15C9df45293925524A8558
> balance: 999.996545434375
> gas used: 1023575 (0xf9e57)
> gas price: 3.375 gwei
> value sent: 0 ETH
> total cost: 0.003454565625 ETH

> Saving artifacts
-----
> Total cost: 0.003454565625 ETH

Summary
-----
> Total deployments: 1
> Final cost: 0.003454565625 ETH

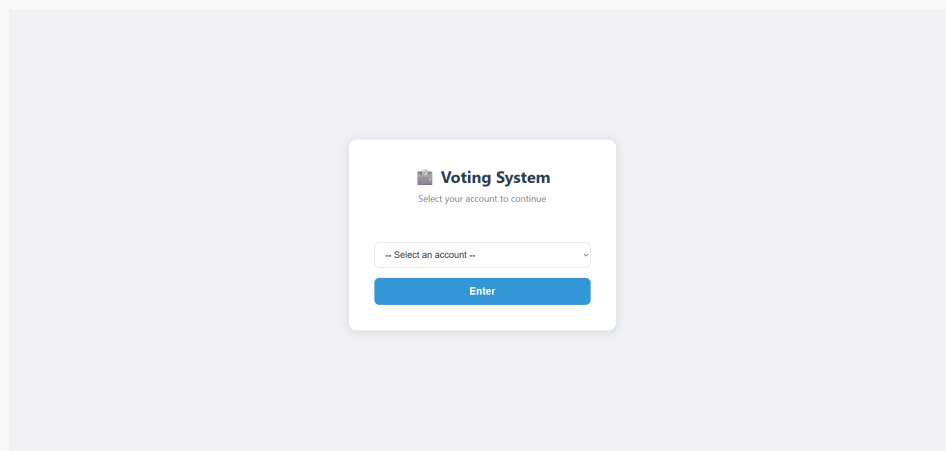
```

4. Interface utilisateur

L'interface est composée de trois pages HTML interconnectées. Chaque page se connecte directement à Ganache via Web3.js sur le port 8545 et charge l'adresse du contrat dynamiquement depuis le fichier Voting.json.

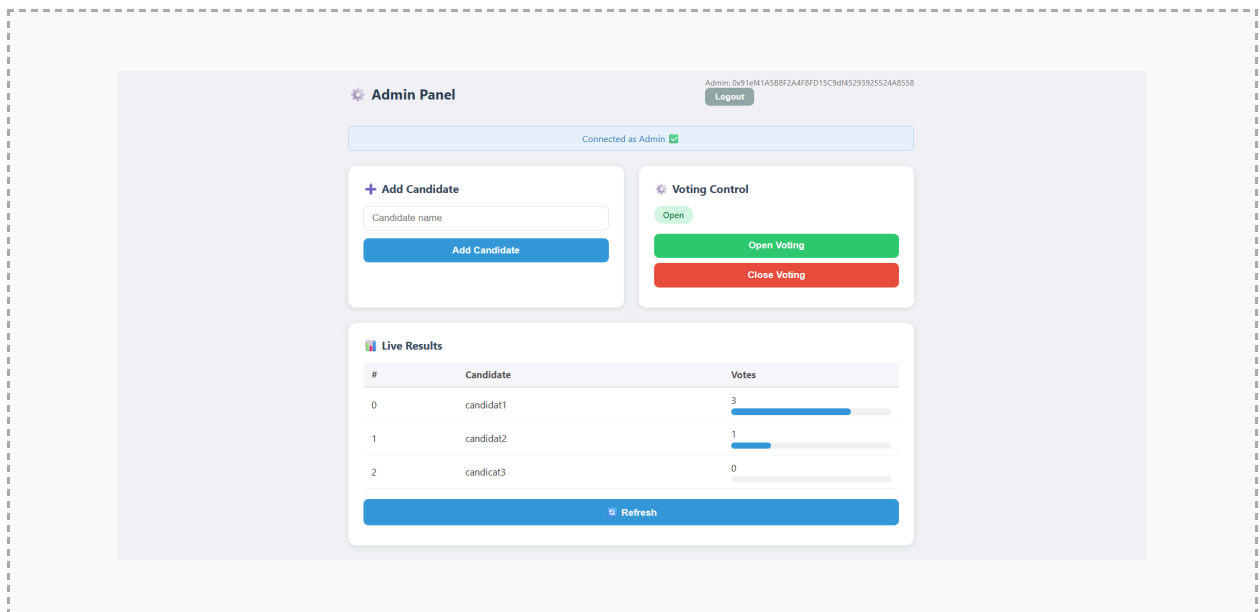
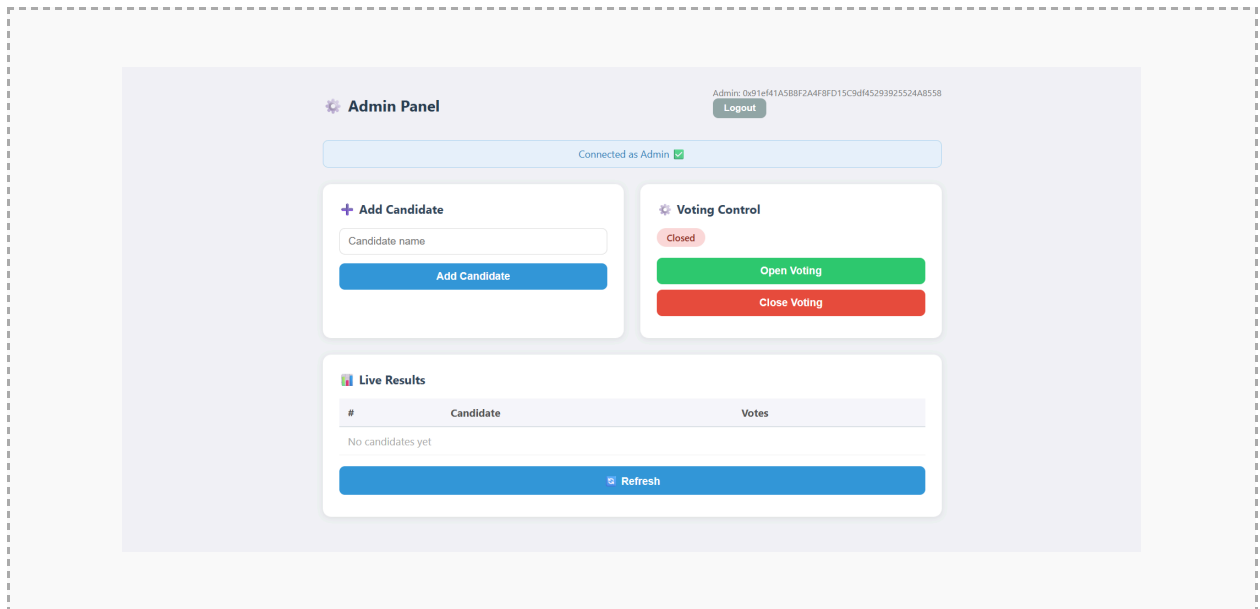
4.1 Page d'accueil (index.html)

Point d'entrée unique de l'application. Elle affiche une liste déroulante de tous les comptes Ganache disponibles. L'utilisateur sélectionne son compte et clique sur "Entrer". Le système détecte automatiquement si le compte est l'administrateur (account[0]) et redirige vers admin.html ou voter.html. La session est maintenue via sessionStorage.



4.2 Panel Administrateur (admin.html)

Accessible uniquement depuis le compte administrateur. Si un électeur tente d'y accéder directement, il est redirigé vers voter.html. Le panel permet d'ajouter des candidats, d'ouvrir/fermer la session de vote, et de visualiser les résultats en temps réel avec des barres de progression proportionnelles au nombre de votes.

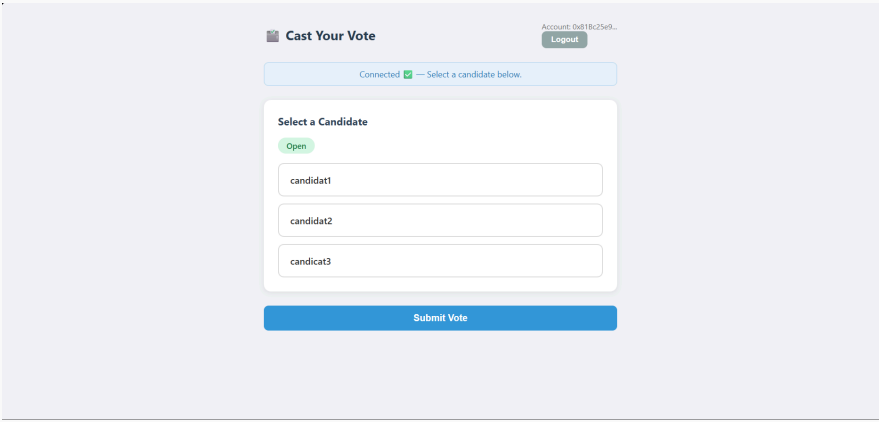


4.3 Page Électeur (voter.html)

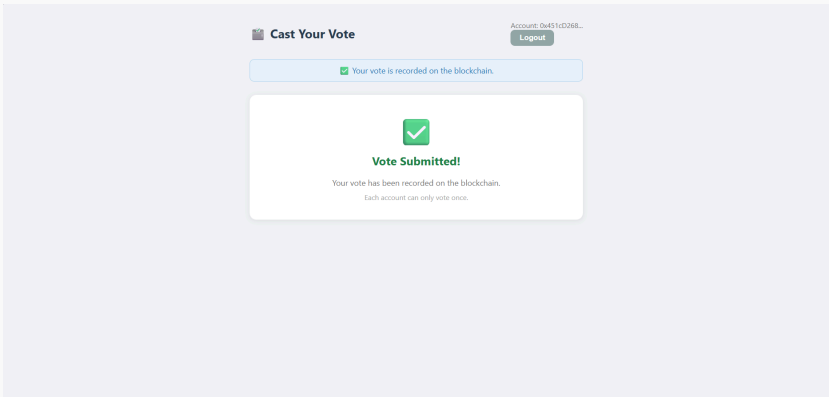
Accessible aux comptes non-administrateurs. La page gère trois états distincts :

- Vote ouvert et électeur n'ayant pas encore voté : affiche les candidats sous forme de cartes cliquables et un bouton de soumission.
- Électeur ayant déjà voté : affiche un message de confirmation (✓) et masque le formulaire. L'état est vérifié directement on-chain via `hasVoted[adresse]`.

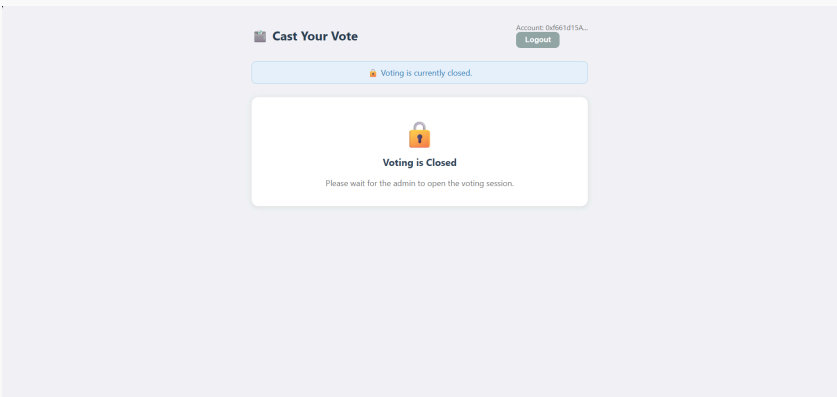
- Session fermée : affiche un message d'attente () invitant l'électeur à patienter.



The screenshot shows the 'Cast Your Vote' interface. At the top right, the user is logged in as 'Account: 0x818a25e9...' with a 'Logout' button. A status bar indicates 'Connected' with a green checkmark and the instruction 'Select a candidate below.' Below this is a 'Select a Candidate' section with an 'Open' button and three input fields labeled 'candidat1', 'candidat2', and 'candidat3'. At the bottom is a blue 'Submit Vote' button.



The screenshot shows the 'Cast Your Vote' interface after a successful vote submission. The status bar now says 'Your vote is recorded on the blockchain.' with a green checkmark. The main content area features a large green checkmark icon, the text 'Vote Submitted!', and a message: 'Your vote has been recorded on the blockchain. Each account can only vote once.'



The screenshot shows the 'Cast Your Vote' interface when the voting session is closed. The status bar indicates 'Voting is currently closed.' with a lock icon. The main content area features a large lock icon, the text 'Voting is Closed', and a message: 'Please wait for the admin to open the voting session.'

5. Tests automatisés

Une suite complète de 19 tests automatisés a été développée dans le fichier test/voting.test.js. Ces tests utilisent le framework Mocha/Chai intégré à Truffle et couvrent l'ensemble des fonctionnalités du smart contract.

Catégorie	Nombre de tests	Ce qui est testé
Deployment	3 tests	Vérification de l'admin, statut initial fermé, zéro candidat au départ
addCandidate	5 tests	Ajout valide, nom stocké, compteur à 0, nom vide rejeté, non-admin rejeté
openVoting / closeVoting	5 tests	Ouverture, fermeture, non-admin rejeté, déjà ouvert rejeté, déjà fermé rejeté
vote	6 tests	Vote valide, marquage hasVoted, double vote rejeté, vote fermé rejeté, index invalide rejeté, décompte multiple correct

Résultat de l'exécution de la suite de tests :

```
node →/workspaces/voting-system (main) $ truffle test --network development
Using network 'development'.

Compiling your contracts...
=====
> Everything is up to date, there is nothing to compile.

Contract: Voting
Deployment
  ✓ should set the deployer as admin
  ✓ should initialize voting as closed
  ✓ should initialize with zero candidates
addCandidate
  ✓ should allow admin to add a candidate (185ms)
  ✓ should store the candidate name correctly (172ms)
  ✓ should initialize candidate vote count to zero (174ms)
  ✓ should reject empty candidate name (171ms)
  ✓ should reject non-admin from adding a candidate
openVoting / closeVoting
  ✓ should allow admin to open voting (54ms)
  ✓ should allow admin to close voting (86ms)
  ✓ should reject non-admin from opening voting
  ✓ should reject opening voting when already open (46ms)
  ✓ should reject closing voting when already closed
vote
  ✓ should allow a voter to cast a vote (59ms)
  ✓ should mark voter as having voted (57ms)
  ✓ should reject double voting (75ms)
  ✓ should reject voting when voting is closed (52ms)
  ✓ should reject invalid candidate index (53ms)
  ✓ should correctly tally votes for multiple voters (179ms)

19 passing (5s)
```

CHAPITRE 4 :

CONCLUSION ET PERSPECTIVES

« Ce chapitre clôture notre rapport de mini projet. Il présente le bilan du travail accompli, les difficultés rencontrées et les perspectives d'évolution envisageables. »

1. Bilan du projet

Au terme de ce mini projet, nous avons atteint l'ensemble des objectifs fixés initialement. Voici le bilan des réalisations :

Module	Statut
Smart contract Voting.sol (Solidity)	✓ Réalisé
Déploiement sur Ganache via Truffle	✓ Réalisé
Interface admin (ajout candidats, contrôle session, résultats)	✓ Réalisé
Interface électeur (vote, confirmation, session fermée)	✓ Réalisé
Page d'accueil avec sélection de compte et redirection	✓ Réalisé
Chargement dynamique de l'adresse du contrat	✓ Réalisé
Suite de tests automatisés (19 tests)	✓ Réalisé — 19/19 passants
Environnement Dev Container Docker	✓ Réalisé
Versionnage GitHub	✓ Réalisé

Ce projet nous a permis de comprendre en profondeur les mécanismes de la Blockchain Ethereum : la notion de transaction, le coût en gas, l'immutabilité des données on-chain, la différence entre appels en lecture (gratuits) et en écriture (transactions payantes en gas), et surtout la puissance des smart contracts pour encoder des règles métier inviolables.

2. Difficultés rencontrées

- Apprentissage de Solidity : le langage présente des comportements spécifiques (gestion du gas, types de données, différence entre storage/memory/calldata) qui diffèrent des langages classiques. Une phase d'apprentissage a été nécessaire avant de commencer le développement.
- Configuration du Dev Container : l'intégration de Ganache et Truffle dans un conteneur Docker a nécessité une configuration précise du forwarding de ports et de l'installation automatique des dépendances.
- Chargement dynamique de l'adresse du contrat : le contrat étant redéployé à chaque redémarrage de Ganache avec une nouvelle adresse, nous avons dû mettre en place un mécanisme de lecture dynamique depuis le fichier Voting.json généré par Truffle.

- Gestion des comptes Ganache : les comptes Ganache changeant à chaque redémarrage, nous avons utilisé un mnémonique fixe pour garantir des adresses stables entre les sessions.

3. Perspectives

Plusieurs évolutions pourraient enrichir ce système :

- Déploiement sur un testnet public (Sepolia) : permettrait de tester le système dans des conditions proches du réseau réel, avec de vrais délais de confirmation de blocs.
- Intégration de MetaMask : remplacer la sélection de compte manuelle par MetaMask, qui est le wallet Ethereum standard pour les DApps, offrirait une expérience plus réaliste.
- Système de liste blanche (whitelist) : ajouter un mécanisme d'enregistrement préalable des électeurs autorisés pour modéliser plus fidèlement un vrai processus électoral.
- Émission d'événements Solidity : utiliser les events Ethereum pour permettre au frontend de recevoir des notifications en temps réel lors de chaque vote, sans nécessiter un rafraîchissement manuel.
- Masquage des résultats jusqu'à la clôture : cacher les décomptes de votes tant que la session est ouverte, pour éviter l'effet de bandwagon lors des élections.

Conclusion

Ce mini projet nous a offert une immersion concrète dans l'écosystème Blockchain Ethereum. Nous avons pu constater par nous-mêmes que la Blockchain n'est pas uniquement une technologie de cryptomonnaie, mais un paradigme de programmation permettant de créer des applications décentralisées aux règles inviolables.

Le système de vote que nous avons développé illustre parfaitement la valeur ajoutée de la Blockchain : la règle "un électeur = un vote" n'est pas simplement une règle d'interface, c'est une règle gravée dans le smart contract, exécutée automatiquement par la machine virtuelle Ethereum, sans possibilité de contournement. C'est cette propriété fondamentale qui distingue une DApp d'une application web classique.

Nous remercions notre encadrant Monsieur Khalid EL KHADIRI pour son accompagnement tout au long de ce projet, et l'EMSI pour nous avoir offert l'opportunité d'explorer ces technologies d'avenir.

Webographie

- Documentation officielle Solidity : <https://docs.soliditylang.org/>
- Documentation Truffle Suite : <https://trufflesuite.com/docs/>
- Documentation Ganache : <https://trufflesuite.com/docs/ganache/>
- Web3.js Documentation : <https://web3js.readthedocs.io/>
- Solidity by Example : <https://solidity-by-example.org/>
- Ethereum Developer Documentation : <https://ethereum.org/en/developers/docs/>
- VS Code Dev Containers : <https://code.visualstudio.com/docs/devcontainers/containers>
- Dépôt GitHub du projet : <https://github.com/NaimaFarhi/Voting-System>